



Phage
SECURITY

SECURITY REVIEW · PREPARED FOR

Bounce Tech

Referrals V2 upgrade

DATE

04.08.2026

CONDUCTED BY

Pyro, Lead Security Researcher
BengalCatBalu, Lead Security Researcher
MarketerKA, Security Researcher

COMMIT

6abcf92

REMEDIATION

1294c37

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK

github.com/phage-security

REACH OUT

Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
8	5	3

SEVERITY DISTRIBUTION



CLIENT	REPOSITORY	METHODS
Bounce Tech	Private	Manual review

CONTRACTS IN SCOPE

src/ReferralsV2.sol	src/LeveragedToken.sol
src/Factory.sol	src/constants/Config.sol

TIMELINE · AUGUST 2026

Audit · 2 days		Remediation · 3 days	
KICK-OFF	END OF AUDIT	REMEDICATIONS START	REMEDICATIONS END
30.07.2026	31.07.2026	03.08.2026	04.08.2026

STARTING COMMIT HASH	6abcf92e19aad12755f318c7e46aabcb96cbec5d
FINAL COMMIT HASH	1294c37d1e6eac31475e56b95c7d3176ab5411ea

03 PROJECT SUMMARY

What Bounce Tech is

Bounce Tech lets users mint and redeem ERC20 leveraged tokens that track an underlying asset with leverage on HyperEVM, paying a redemption fee that the referrals layer partly rebates to referral codes.

This review covers the Referrals V2 upgrade. A wallet that clears a volume threshold generates its own referral code, referees bind to a code, and each redemption routes a share of the fee back to the referrer and referee. The owner migrates the v1 codes and bindings and can grant elevated affiliate splits.

SECURITY POSTURE

BEFORE REMEDIATION

6abcf92

Code ownership and migration

M-01 · M-02 · L-01 · L-05

Migration and self-serve creation resolved codes by string at execution time and left the code slot contestable. A case-folded string, a launch-window race, or an out-of-spec migrated code could seat the wrong referrer or block the intended one.

Fee and volume accounting

L-02 · L-03

Rebates and volume were computed on the wrong base or credited to the wrong party. The flat execution fee was fed into the rebate, and volume was recorded to `msg.sender` rather than the funded user.

Silent binding decay

L-04 · L-06

A bound referee kept paying fees with no rebate and no on-chain signal after its code was removed or its split dropped to zero, with no self-serve way out.

AFTER REMEDIATION

1294c37

Five of eight findings fixed and verified.

`migrateBinding` now takes the referrer address directly, so two v1 codes differing only in case can no longer fold onto one referrer. The migration script sets `unlockThreshold` after seeding, closing the launch-window squat, and `migrateReferrer` now clears an occupied code and seats the partner on it, closing the advertised-code grab. `mint` and `redeem` credit volume to the funded recipient, and `_migrateReferrer` runs migrated codes through the same charset and length validation as self-serve codes.

The three acknowledged items are documented trade-offs, none a loss of principal: the flat `executeRedemptionFee` still flows into the rebate under a maxed affiliate split, a removed leveraged token keeps serving redemptions with no rebate or recorded volume, and a referee whose split is dropped to zero has no on-chain unbind. Each is operationally containable and left as accepted behavior.

04 RECOMMENDATIONS

Beyond this audit

Four measures that extend the protection of this review after launch, independent of any individual finding.

R-01	Bug bounty	Stand up a public bounty on Immunefi or HackenProof with the top tier covering fund loss.
R-02	Invariant monitoring	Run an off-chain watcher that reconciles <code>ReferralsV2</code> state and pages on divergence. Check that base-asset balance held covers the sum of unclaimed <code>rebates</code> , that <code>codeReferrers</code> and <code>referrerCodes</code> stay bijective, and that no wallet below <code>unlockThreshold</code> holds a self-serve code.
R-03	Key hygiene	The <code>GlobalStorage</code> owner is the highest-value key in this system. Keep it on a multisig with hardware signers.
R-04	Partner onboarding runbook	<code>migrateReferrer</code> now seats a partner on an occupied code by clearing the holder's slot. Accrued rebates stay claimable and the referees bound to the displaced holder are not moved, so check both on every seating, and set the split deliberately: the overwrite rewrites it and emits no <code>SetAffiliate</code> .

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
src/ReferralsV2.sol	New UUPS-upgradeable referral registry. Owner migrates v1 codes and bindings and grants affiliate splits, while wallets that clear <code>unlockThreshold</code> self-serve generate codes. Rebates are funded per redemption by the leveraged tokens.
src/LeveragedToken.sol	Records referral volume on mint and redeem and routes redemption-fee rebates through the referrals layer, wrapping both calls in try/catch so referrals never block a mint or redeem.
src/Factory.sol	Marks addresses in <code>ltExists</code> , the caller gate that <code>recordVolume</code> and <code>donateRebates</code> enforce. <code>_removeLt</code> clears the flag once perp margin is settled.

IReferralsV2.sol reviewed as reference for ReferralsV2.sol, not counted in nSLOC. For existing contracts only the PR #216 diff is in scope.

PRIVILEGED ROLES

Protocol owner. The `GlobalStorage` owner is the sole `onlyOwner` on `ReferralsV2`. Controls affiliate splits, code revocation and migration, historic-volume seeding, `unlockThreshold`, base bps, and UUPS upgrades.

Leveraged tokens. Any address the `Factory` marks in `ltExists` may call `recordVolume` and `donateRebates`. Deployed only by the factory.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	<code>migrateBinding</code> takes no expected-referrer argument	MEDIUM	● FIXED
M-02	Migration is not atomic and <code>unlockThreshold</code> is set before seeding	MEDIUM	● FIXED
L-01	Anyone can take a referral code the protocol advertises	LOW	● FIXED
L-02	Referral rebate is taken out of the flat <code>executeRedemptionFee</code>	LOW	● ACKNOWLEDGED
L-03	Volume is credited to <code>msg.sender</code> , so integrators absorb their users' volume	LOW	● FIXED
L-04	A removed leveraged token still redeems but pays no rebate and records no volume	LOW	● ACKNOWLEDGED
L-05	Migrated codes bypass the code format rules	LOW	● FIXED
L-06	A referee cannot leave a referrer when <code>refereeBps</code> is set to 0	LOW	● ACKNOWLEDGED

● MEDIUM SEVERITY · 2 FINDINGS

M-01

MEDIUM

FIXED

migrateBinding takes no expected-referrer argument

v1 stored every binding as a referrer address, so the export already names each referrer. `_migrateBinding` did not take it. It took a code string and resolved the referrer at execution time, and v2 lowercases the string while v1 never did.

```
string memory norm_ = _toLower(referralCode_);
address referrer_ = $.codeReferrers[norm_];
$.refereeReferrers[referee_] = referrer_;
```

Two v1 codes differing only in case fold to one v2 code. Referrer migration catches this: it refuses a code someone already holds. Binding migration cannot, it has no referrer to compare against, so it writes whoever holds the folded code and reverts nothing.

1. v1 has `Alpha` owned by A and `alpha` owned by B
2. A migrates first on a 1000/400 split, so `alpha` in v2 belongs to A
3. B's row reverts `CodeAlreadyExists`, the operator moves B to a free code
4. carol joined B under `alpha`, so `migrateBinding(carol, "alpha")` binds her to A with no revert
5. on a 50,000 USDC redemption of a 3x token carrying a 450 USDC fee, A takes the 45 USDC referrer leg that belonged to B

RECOMMENDATION

Take the referrer address and drop the string. v1 already holds it, the string only carries the fold.

```
function _migrateBinding(address referee_, string calldata referralCode_) internal {
+   function _migrateBinding(address referee_, address referrer_) internal {
+       if (referee_ == address(0)) revert InvalidReferrer();
+       if (referrer_ == address(0)) revert InvalidReferrer();
+       if (referrer_ == referee_) revert CannotReferSelf();
string memory norm_ = _toLower(referralCode_);
address referrer_ = $.codeReferrers[norm_];
if (referrer_ == address(0)) revert CodeDoesNotExist();
if (referrer_ == referee_) revert CannotReferSelf();
+       string memory norm_ = $.referrerCodes[referrer_];
+       if (bytes(norm_).length == 0) revert CodeDoesNotExist();
```

M-02

MEDIUM

FIXED

Migration is not atomic and `unlockThreshold` is set before seeding

`ReferralsV2Migration._run` configures the contract before it seeds it. `_configure` sets `unlockThreshold` first, and the run is broadcast as independent transactions, not one atomic call.

```
function _configure(ReferralsV2 referrals, uint256 scale_) internal {
    referrals.setUnlockThreshold(Config.UNLOCK_THRESHOLD * scale_);
    referrals.setBaseReferrerBps(Config.BASE_REFERRER_BPS);
    referrals.setBaseRefereeBps(Config.BASE_REFEREE_BPS);
}
```

`generateCode` gates on `unlockThreshold` being non zero and the caller's recorded volume.

```
if ($.unlockThreshold == 0 || $.volume[msg.sender] < $.unlockThreshold) revert
VolumeThresholdNotMet();
```

Commit 03a33cc added the `unlockThreshold == 0` clause to block squatting on a fresh proxy. Setting the threshold in `_configure` spends that protection before a single code is migrated. Once `setHistoricVolume` lands, every seeded wallet clears both gates while `migrateReferrers` is still pending. v1 codes are public, so the target list needs no pending calldata and a private mempool does not close the window.

1. `setUnlockThreshold(100_000e6)` confirms
2. `setHistoricVolume` confirms, crediting seeded traders their v1 volume
3. Seeded trader T reads a live v1 code and calls `generateCode("bouncegm")`
4. `migrateReferrers` lands and reverts on that entry
5. T holds the code, everyone following the published link binds to T

RECOMMENDATION

Set `unlockThreshold` after seeding and migration complete. While it stays zero, `generateCode` reverts for every caller.

```
_configure(referrals, scale_);
_seed(referrals);
+ referrals.setUnlockThreshold(Config.UNLOCK_THRESHOLD * scale_);

globalStorage.setReferrals(address(referrals));
```

```
function _configure(ReferralsV2 referrals, uint256 scale_) internal {
- referrals.setUnlockThreshold(Config.UNLOCK_THRESHOLD * scale_);
    referrals.setBaseReferrerBps(Config.BASE_REFERRER_BPS);
}
```

● LOW SEVERITY · 6 FINDINGS

L-01

LOW

FIXED

Anyone can take a referral code the protocol advertises

`generateCode` is permissionless once a wallet clears the volume threshold, and the owner's own seating path guards the slot with the identical check, so on a contested string it is the owner's call that reverts.

```
if ($.codeReferrers[norm_] ≠ address(0)) revert CodeAlreadyExists(); // generateCode
if ($.codeReferrers[norm_] ≠ address(0)) revert CodeAlreadyExists(); // _migrateReferrer
```

Partner codes are brand names, so they can be parked speculatively before a partner exists, and `migrateReferrer` carries no deadline. Clearing the threshold costs about 150 USDC in redemption fees. On a 50,000 USDC round trip at 3x the holder collects 45 USDC of the 450 USDC fee. `revokeCode` plus a reseal takes the string back, but accrued rebates stay claimable and the referees bound in the meantime cannot be moved by the owner.

RECOMMENDATION

Let `migrateReferrer` take a code someone already holds instead of reverting on it. That is simpler than closing code creation before every handover, and the two residuals above survive either way.

Taking the code over also rewrites its split. That is the storage `setAffiliate` writes, and it is what a bound referee's rebate is read from, so emit `SetAffiliate` next to `MigrateReferrer`. Skip the `RevokeCode` when the code is re-seated on the referrer who already holds it: nothing is revoked there, but the split still changes and every referee bound to that referrer is repriced with no event.

L-02

LOW

ACKNOWLEDGED

Referral rebate is taken out of the flat `executeRedemptionFee`

`executeRedemptionFee` is a flat fee charged only on the executor-driven redemption path, where a third party spends gas to process a queued redemption. `_redemptionFee` folds it into the total and the whole sum is forwarded to `donateRebates`.

```
uint256 percentFee_ = baseAmount_.mul(globalStorage_.redemptionFee()).mul($.targetLeverage);
if (!hasFlatFee_) return percentFee_;
return percentFee_ + globalStorage_.executeRedemptionFee();
```

```
try referrals_.donateRebates(beneficiary_, feeAmount_) returns (uint256 rebated_) {
    if (rebated_ ≤ feeAmount_) feeAmount_ -= rebated_; // rebate off the top of percent +
    flat
} catch {}
_payFees(feeAmount_); // treasury receives only the remainder
```

`donateRebates` computes both slices on the full `feeAmount_`, so the rebate is taken from `percentFee_ + executeRedemptionFee`, not the percent alone. `setAffiliate` caps splits only at `referrerBps_ + refereeBps_ ≤ 10_000`. Under a maxed split `totalRebate_` equals the entire fee, leaving zero for `_payFees`.

RECOMMENDATION

Exclude `executeRedemptionFee` from the rebate base: rebate only `percentFee_` and route the flat component straight to `_payFees`.

L-03

LOW

FIXED

Volume is credited to `msg.sender`, so integrators absorb their users' volume

`mint` sends the minted tokens to `to_` but records referral volume against `msg.sender`, and `redeem` does the same. When the caller is an intermediary routing flow on behalf of its users, the volume accrues to the integrator contract, not the user who supplied the capital.

```
_mint(to_, ltAmount_);  
_recordVolume(msg.sender, baseAmount_); // credited to caller, not to_
```

```
_baseAsset().safeTransfer(to_, afterFees_);  
_recordVolume(msg.sender, baseAmount_); // credited to caller, not to_
```

The `mint` path already separates recipient from caller, so volume attribution is a distinct axis the interface did not expose. Any protocol routing user flow through one contract collects all the volume in one address.

RECOMMENDATION

Add an explicit volume beneficiary parameter, or reuse `to_`, so credited volume can be directed to the actual user rather than always `msg.sender`.

L-04

LOW

ACKNOWLEDGED

A removed leveraged token still redeems but pays no rebate and records no volume

`recordVolume` and `donateRebates` both gate on `factory().ltExists(msg.sender)` and revert with `NotLeveragedToken` once the caller is deregistered. The leveraged token swallows both reverts, so the redemption itself never fails.

```
if (!_GLOBAL_STORAGE.factory().ltExists(msg.sender)) revert NotLeveragedToken();
```

```
try IReferralsV2(...).recordVolume(user_, baseAmount_.mul($.targetLeverage)) {} catch {}  
// ...  
try referrals_.donateRebates(beneficiary_, feeAmount_) returns (uint256 rebated_) {  
    if (rebated_ ≤ feeAmount_) feeAmount_ -= rebated_;  
} catch {}
```

`Factory._removeLt` clears `ltExists` while blocking only on leftover perp margin, not on outstanding supply or pending redemptions.

```
if (perpUsdc_ > 0) revert StillHasMargin();  
// ...  
delete $.ltExists[ltAddress_]; // holders can still redeem afterwards
```

RECOMMENDATION

Track the deregistered token's volume and reapply it via `setHistoricVolume`, or block removal while redeemable supply remains.

L-05

LOW

FIXED

Migrated codes bypass the code format rules

`generateCode` routes through `_normalizeAndValidate`, which enforces a 3 to 15 byte length and a `[a-z0-9]` charset. `_migrateReferrer` used `_toLowerCase` instead, which only lowercases ASCII letters and applies no length or charset check.

```
string memory norm_ = _toLowerCase(referralCode_);  
if (bytes(norm_).length == 0) revert InvalidReferralCode();
```

v1 allowed out-of-spec codes to exist, because `Referrals.addReferrer` rejected only the empty string. Any v1 code could therefore be migrated with punctuation, whitespace, or multi-byte sequences at any non-zero length, while staying usable for lookup, binding, and rebate accrual.

RECOMMENDATION

Validate migrated codes against the same rules as self-serve codes, and re-issue by hand any v1 code that fails.

L-06

LOW

ACKNOWLEDGED

A referee cannot leave a referrer when `refereeBps` is set to 0

Once a referee is bound, `joinWithReferral` is the only rebinding path and it reverts while the current referrer's code still exists. There is no way for the referee to reset `refereeReferrers[msg.sender]`.

```
address existingReferrer_ = $.refereeReferrers[msg.sender];
if (existingReferrer_ != address(0) && bytes($.referrerCodes[existingReferrer_]).length != 0)
{
    revert UserAlreadyJoined();
}
```

`setAffiliate` can drop `refereeBps` to 0, since it only bounds the sum.

```
if (uint256(referrerBps_) + refereeBps_ > _BPS_DENOMINATOR) revert InvalidSplit();
$.codeSplits[norm_] = Split({referrerBps: referrerBps_, refereeBps: refereeBps_, isAffiliate:
true});
```

A referee who joined under a favorable `refereeBps` can have it dropped to zero and cannot react. The only release is `revokeCode`, which is `onlyOwner`, and rebinding is race-conditional: the referrer can mint a new code in the same block.

RECOMMENDATION

Allow a referee to unbind or switch when the split moves materially against them, for example a referee-callable unbind or rebinding permitted when `refereeBps` drops below a threshold. Otherwise document that the affiliate split is owner-mutable after binding so referees can assess the risk before joining.